
Université Abdelmalek Essaâdi

Faculté des Sciences et Techniques – Tanger

Département Informatique

Licence : Analytique des Données

Miniprojet 2

Data Lake & Analyse du Marché de l'Emploi IT au Maroc

Mexora RH Intelligence

Réalisé par :

Ouissal SEKKARI

Abir FATAH

Encadré par :

M. Hassan ZILI

Document de Conception — Architecture Data Lake

Justification des Formats par Zone

Zone	Format	Pourquoi ce format ?	Pourquoi pas les autres ?
Bronze	JSON	Les offres d'emploi viennent de sources semi-structurées (API, scraping). JSON est le format natif de ces sources : zéro transformation à l'ingestion, fidélité totale au schéma source. Conforme au principe schema-on-read : le schéma sera appliqué seulement à la lecture en zone Silver.	CSV forcerait un aplatissement des champs imbriqués (ex : liste de compétences) et une perte d'information. Parquet exigerait un schéma fixe avant chargement, ce qui est incompatible avec la zone brute.
Silver	Parquet	Parquet est un format colonnaire compressé (Snappy). Il réduit significativement l'espace disque, accélère les requêtes analytiques DuckDB sur grandes colonnes (ex : salaire, compétences), et supporte le partitionnement natif par dossier.	CSV est volumineux, sans typage fort et lent à lire en colonnaire. JSON reste valable mais perd l'avantage de la compression et de la lecture colonnaire nécessaires pour la zone Silver.
Gold	Parquet	Même logique que Silver, avec en plus la cohérence d'un seul format pour les outils analytiques aval (DuckDB, Metabase). Les agrégats Gold sont de petits volumes fortement lus : Parquet offre le meilleur ratio taille/-vitesse.	Une base de données (SQLite, PostgreSQL) serait possible mais introduit une dépendance infrastructure. Parquet reste portable, lisible par n'importe quel outil Python ou SQL sans serveur.

Questions de Conception

2.1 Pourquoi conserver les données brutes en zone Bronze sans les modifier ?

La zone Bronze est le **contrat de confiance** entre les données sources et le système d'analyse. Elle doit être conservée intacte pour plusieurs raisons fondamentales :

- **Reproductibilité totale** : Si une transformation Silver introduit une erreur (ex : mauvaise conversion de devise), on repart des données Bronze sans relancer le scraping. C'est le filet de sécurité du pipeline.

- **Traçabilité légale et métier** : En cas d’audit (« pourquoi cette offre a-t-elle un salaire de 0 MAD ? »), Bronze prouve exactement ce qui a été reçu depuis la source.
- **Évolution des règles métier** : Les règles Silver évoluent (nouveau taux EUR/MAD, nouvelle ville à mapper). Avec Bronze intact, on peut re-transformer sans repasser par le scraping.
- **Risques si absent** : Perte irrémédiable de l’information originale, impossibilité de distinguer un bug de pipeline d’une erreur de source, risque de *data drift* non détectable.

2.2 Schema-on-read vs Schema-on-write

Critère	Schema-on-write (DWH classique)	Schema-on-read (Data Lake)
Définition	Le schéma est défini avant l’écriture. Les données doivent se conformer au schéma à l’ingestion.	Le schéma est appliqué au moment de la lecture. Les données sont stockées telles quelles.
Exemple	Table SQL offres(salaire DECIMAL(10,2)) : impossible d’insérer "Selon profil".	JSON Bronze : "salaire_brut": "Selon profil" stocké tel quel. Conversion effectuée en Silver.
Avantage	Données toujours cohérentes, requêtes simples.	Ingestion rapide, flexibilité totale, supporte les schémas variables.
Inconvénient	Rigide : un nouveau champ exige une migration de schéma coûteuse.	Risque de mauvaise qualité si Silver est mal conçue.
Usage idéal	Données structurées stables (transactions bancaires).	Données semi-structurées variées (offres d’emploi scrapées).

2.3 Choix de partitionnement par zone

Bronze : partitionnement par source / mois

Le partitionnement Bronze reflète la **provenance** des données. Chaque source (Rekrute, MarocAnnonce, LinkedIn) a ses propres caractéristiques de format et de qualité. En cas de problème sur une source spécifique, on peut isoler et réingérer uniquement cette partition. Le sous-partitionnement par mois permet de réingérer une période précise indépendamment des autres.

bronze/rekrute/2024_01/offres_raw.json	<- 194 offres Rekrute janvier 2024
bronze/linkedin/2024_01/offres_raw.json	<- offres LinkedIn du meme mois

Silver : partitionnement par ville / mois

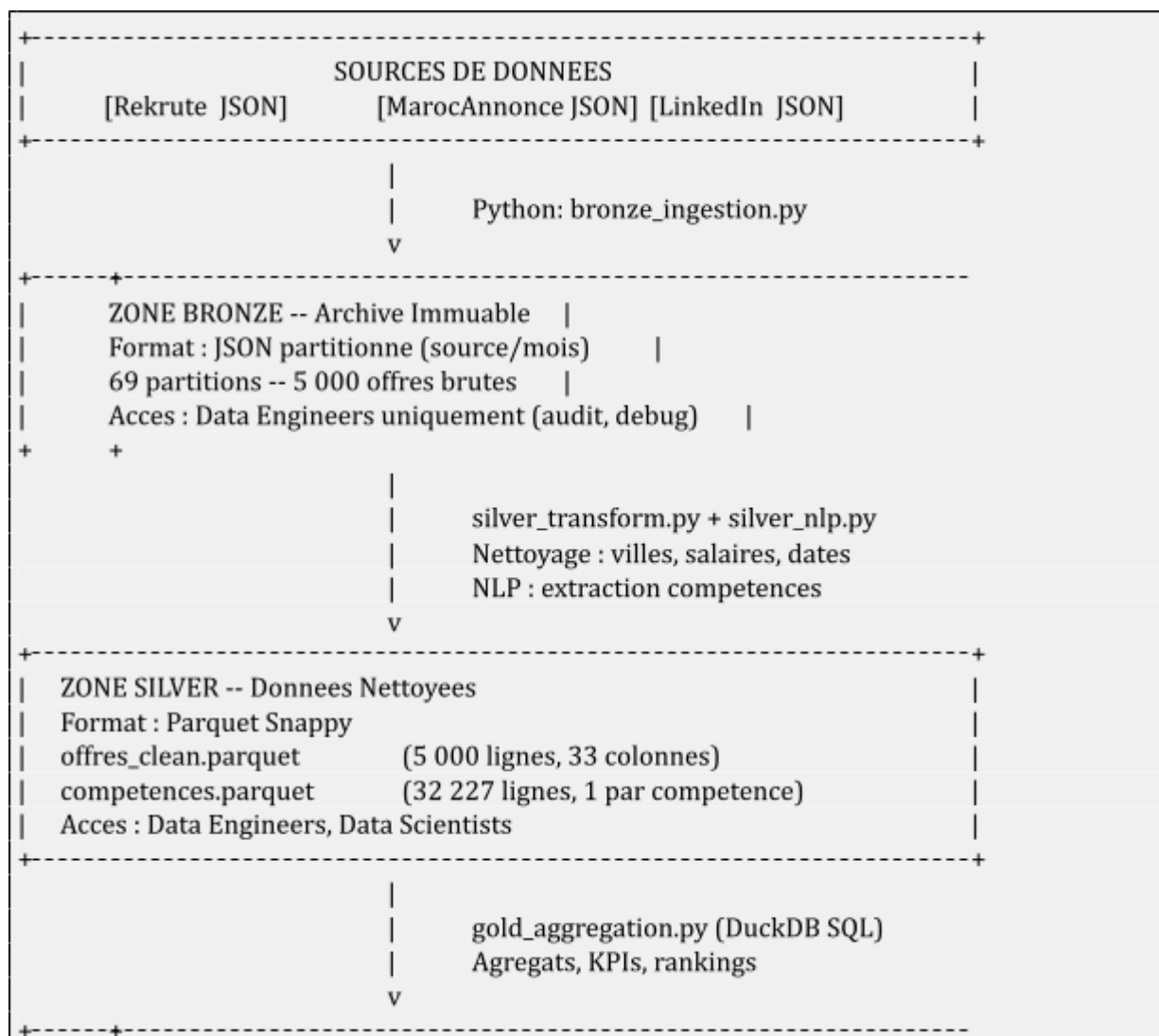
En Silver, le partitionnement devient **analytique**. Les requêtes les plus fréquentes filtrent par ville (Tanger vs Casablanca) et par période. DuckDB effectue du *partition pruning* : pour WHERE ville = 'Tanger' AND mois = '2024-06', seuls les fichiers de cette partition sont lus.

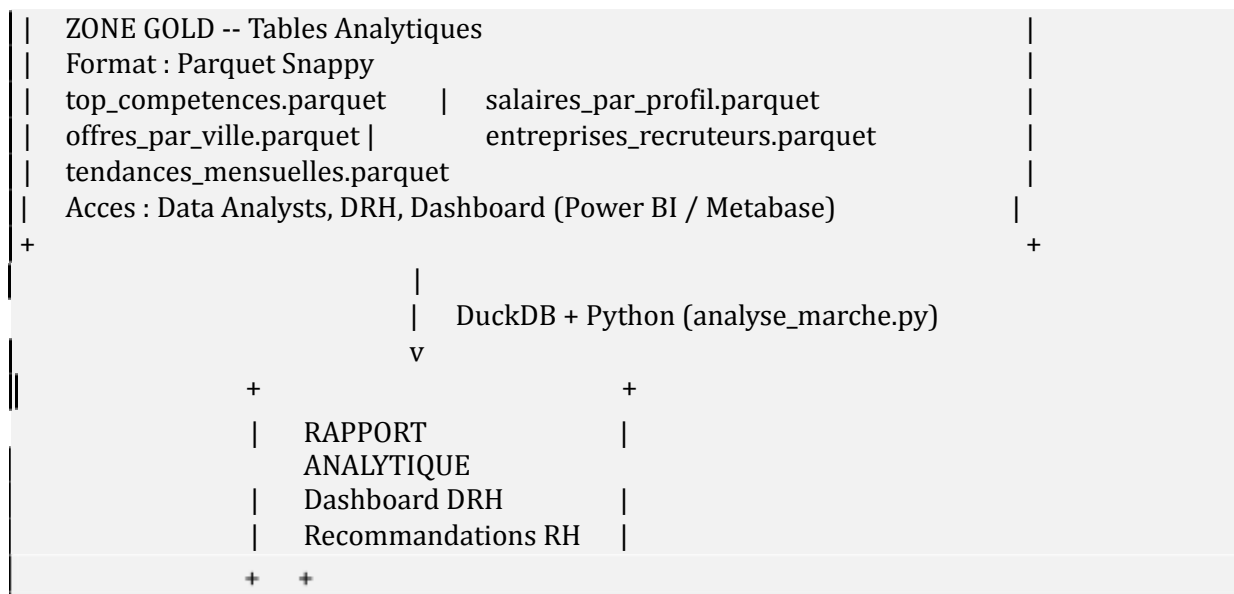
2.4 Gouvernance : éviter le Data Swamp

Un Data Lake non gouverné devient un **Data Swamp** : données non documentées, non traçables, inutilisables. Les règles suivantes sont appliquées :

- **Règle 1 — Catalogue** : Chaque fichier Bronze embarque des métadonnées (date_ingestion, source, nb_offres, partition). Le rapport_pipeline.md documente toutes les transformations.
- **Règle 2 — Immuabilité Bronze** : Aucun processus ne modifie les fichiers Bronze. Toute correction passe obligatoirement par Silver.
- **Règle 3 — Nommage standardisé** : offres_raw.json en Bronze, offres_clean.parquet en Silver, noms explicites en Gold (salaires_par_profil.parquet).
- **Règle 4 — Qualité mesurée** : Chaque étape Silver loggue ses métriques (% salaires connus, % dates valides, % titres classifiés). Un seuil d'alerte à 70% déclenche une revue manuelle.
- **Règle 5 — Versioning Git** : Tous les scripts sont versionnés sur GitHub. Chaque modification de règle métier génère un commit documenté.

Schéma d'Architecture





Outils utilisés par étape :

- Bronze → Silver : Python 3.11, pandas, re (regex)
- Silver → Gold : DuckDB, pandas, pyarrow
- Gold → Analyse : DuckDB, matplotlib, seaborn, plotly
- Stockage : système de fichiers local (simulant un objet store S3/GCS)